dbt

The Complete Guide

From Scratch to Data Hero

Analytics Engineering

Modern Data Stack

SQL · Git · Testing

What's Inside

- **15 Chapters** — From installation to production CI/CD
- **All Materializations** — Views, Tables, Incrementals, Ephemeral
- **Testing & Quality** — Generic, singular, dbt-expectations
- **Jinja & Macros** — DRY SQL with reusable functions
- **Snapshots** — SCD Type 2 history tracking
- **Advanced Patterns** — Contracts, unit tests, orchestration

Data Build Tool (dbt) · docs.getdbt.com · hub.getdbt.com

Table of Contents

CHAPTER 1

dbt (Data Build Tool) is an open-source command-line tool that lets analysts and engineers transform data in their warehouse by writing SELECT statements. It handles the **T** in **ELT** (Extract, Load, Transform). dbt does not move data — it transforms data that is already loaded into your warehouse.

Before dbt, analysts wrote ad-hoc SQL scripts with no version control, no testing, no documentation, and no dependency management. One broken query could cascade silently through dashboards. dbt solves all of this.

The ELT Architecture

Stage	Tool Examples	What Happens
Extract + Load	Fivetran, Airbyte, Stitch	Raw data moved into warehouse
Transform (dbt)	dbt Core / dbt Cloud	SQL transforms on top of raw data
Serve	Looker, Tableau, Metabase	BI tools query clean mart tables

What dbt Solves

Version Control	All transformations live in Git — full history, PRs, collaboration.
Built-in Testing	Test uniqueness, not-null, accepted values, and referential integrity.
Auto Documentation	Generate a searchable data catalog from YAML + SQL comments.
Modularity (ref)	Reference other models with ref(). No copy-paste SQL. DAG auto-built.
DAG Lineage	Visual dependency graph from raw source to BI layer.
Reproducibility	Same code → same results. Idempotent by design.
Dev/Prod Isolation	Each developer gets their own schema. No stepping on each other.
Incremental Loads	Process only new rows on large tables. Huge performance win.

The Analytics Engineering Role

Role	Focus	Tools
Data Engineer	Pipelines, infra, raw ingestion	Spark, Kafka, Airflow
Analytics Engineer	Transformation layer, data modeling	dbt, SQL, Git
Data Analyst	Insights, dashboards, ad-hoc analysis	Looker, Tableau, Python

Supported Warehouses (via Adapters)

Warehouse	Adapter Package	Best For
Snowflake	dbt-snowflake	Enterprise, multi-cloud
BigQuery	dbt-bigquery	Google Cloud native
Redshift	dbt-redshift	AWS native
Databricks	dbt-databricks	Lakehouse / Spark
DuckDB	dbt-duckdb	Local dev, embedded analytics
Postgres	dbt-postgres	Self-hosted, open source
Trino/Presto	dbt-trino	Federated queries
ClickHouse	dbt-clickhouse	OLAP, real-time analytics

CHAPTER 2

Prerequisites

Python 3.8+, pip or pipx, a data warehouse (or DuckDB for local dev), Git.

Install dbt Core + Adapter

```
# Install dbt + your warehouse adapter (choose one) pip install dbt-core dbt-snowflake # Snowflake pip install dbt-core dbt-bigquery # BigQuery pip install dbt-core dbt-redshift # Redshift pip install dbt-core dbt-duckdb # DuckDB (local dev, no cloud needed) pip install dbt-core dbt-databricks # Databricks # Using pipx (recommended - isolates environment) pipx install dbt-core # Verify installation dbt --version
```

TIP

Use pipx install dbt-core to install dbt in its own isolated virtual environment. This avoids dependency conflicts with other Python packages in your project.

Initialize Your First Project

```
# Create a new dbt project (interactive wizard) dbt init my_project # You will be prompted: adapter type, account, credentials cd my_project # Test the warehouse connection dbt debug
```

Project Directory Structure

```
my_project/ ■■■ dbt_project.yml # Project config: name, version, model paths ■■■ packages.yml # External package dependencies ■■■ models/ # All your SQL transformation files ■ ■■■ staging/ # 1:1 with sources — light cleaning only ■ ■ ■■■ _sources.yml # Source declarations ■ ■ ■■■ _schema.yml # Staging model docs + tests ■ ■ ■■■ stg_orders.sql ■ ■ ■■■ stg_customers.sql ■ ■■■ intermediate/ # Business logic joins ■ ■ ■■■ int_orders_enriched.sql ■ ■■■ marts/ # Final tables for BI tools ■ ■■■ _schema.yml ■ ■■■ fct_orders.sql ■ ■■■ dim_customers.sql ■■■ tests/ # Custom singular SQL tests ■■■ snapshots/ # SCD Type 2 history tracking ■■■ seeds/ # CSV lookup tables ■■■ macros/ # Reusable Jinja functions ■■■ analyses/ # Ad-hoc SQL (not materialized) ■■■ target/ # Compiled SQL output (git-ignored)
```

The profiles.yml File

Lives at **~/.dbt/profiles.yml**. This file contains your warehouse credentials. **Never commit this to Git.** Use environment variables for secrets.

```
my_project: target: dev outputs: dev: type: snowflake account: abc12345.us-east-1 user: your_user password: "{{ env_var('DBT_PASSWORD') }}" role: transformer database: analytics warehouse: compute_wh schema: dbt_yourname # personal dev schema threads: 4 prod: type: snowflake account: abc12345.us-east-1 user: dbt_prod_user password: "{{ env_var('DBT_PROD_PASSWORD') }}" role: transformer database: analytics warehouse: compute_wh schema: public # production schema threads: 8
```

The dbt_project.yml File

```
name: 'my_project' version: '1.0.0' profile: 'my_project' model-paths: ["models"] seed-paths:
["seeds"] test-paths: ["tests"] snapshot-paths: ["snapshots"] macro-paths: ["macros"]
target-path: "target" clean-targets: ["target", "dbt_packages"] models: my_project: staging:
+materialized: view # all staging models = views +schema: staging intermediate: +materialized:
ephemeral # not stored in DB, used as CTEs marts: +materialized: table # all mart models =
physical tables finance: +tags: ['finance', 'daily'] +post_hook: "GRANT SELECT ON {{ this }} TO
ROLE reporter"
```

Essential dbt Commands

Command	What It Does
dbt run	Execute all models (create views/tables)
dbt test	Run all tests
dbt build	Run + Test + Seed + Snapshot in DAG order (recommended)
dbt compile	Compile Jinja to raw SQL without running
dbt docs generate	Generate documentation site
dbt docs serve	Open docs at localhost:8080
dbt seed	Load CSV files to warehouse
dbt snapshot	Run SCD Type 2 snapshots
dbt source freshness	Check if raw data is up to date
dbt deps	Install packages from packages.yml
dbt debug	Test warehouse connection
dbt run -s model_name	Run a specific model
dbt run -s +model_name	Run model + all its ancestors (parents)
dbt run -s model_name+	Run model + all its descendants (children)
dbt run -s tag:finance	Run all models tagged 'finance'
dbt run --full-refresh	Rebuild incremental models from scratch
dbt run --target prod	Run against production profile
dbt run --exclude model_x	Run all except model_x

CHAPTER 3

The DAG (Directed Acyclic Graph)

dbt automatically builds a dependency graph of all your models based on ref() calls. This DAG determines execution order — no manual orchestration needed. dbt guarantees models execute in the correct order and in parallel where possible.

```
# Visualize the DAG dbt docs generate && dbt docs serve # Then open the lineage graph in the browser # Visualize in terminal dbt ls --select +fct_orders # list all ancestors of fct_orders
```

Materializations

Type	Creates In Warehouse	Best For	Freshness
view	A database view	Staging, lightweight transforms	Always current
table	Full physical table rebuild	Mart tables, BI consumption	Rebuilt each run
incremental	Appends/merges only new rows	Large event tables, logs	Near real-time
ephemeral	CTE — not stored in DB	Intermediate logic, reuse w/o storage	N/A
materialized view	DB-native materialized view	Auto-refreshed views (warehouse dep)	DB managed

The ref() Function — Most Important Function in dbt

ref('model_name') is how models depend on each other. It automatically builds the DAG, resolves to the correct schema/database in dev vs prod, and enables parallel execution.

```
-- models/marts/fct_orders.sql SELECT o.order_id, o.order_date, c.customer_name, c.customer_segment, o.total_amount FROM {{ ref('stg_orders') }} o -- references staging model JOIN {{ ref('stg_customers') }} c -- dbt auto-resolves schema ON o.customer_id = c.customer_id WHERE o.status = 'completed'
```

The source() Function

source('source_name', 'table_name') references raw tables loaded by ingestion tools. It enables source freshness checks and shows lineage from the very first raw table.

```
-- Only used in staging models (first layer touching raw data) SELECT * FROM {{ source('salesforce', 'accounts') }}
```

Naming Conventions (Best Practice)

Layer	Prefix	Example	Purpose
Staging	stg_	stg_salesforce__accounts	1:1 with source, rename/cast only
Intermediate	int_	int_orders_with_items	Business logic, joins
Fact table	fct_	fct_orders	Transactional events

Layer	Prefix	Example	Purpose
Dimension	dim_	dim_customers	Descriptive attributes
Aggregated	agg_	agg_daily_revenue	Pre-aggregated summaries
Utility	util_	util_date_spine	Helper models

For staging models, use double underscore to separate source name from table: stg___. For example: stg_salesforce__opportunities.

CHAPTER 4

A model is a .sql file containing a SELECT statement. dbt wraps it in CREATE TABLE AS or CREATE VIEW AS based on its materialization. Every model is a node in the DAG.

1. Staging Models

One-to-one with source tables. Light cleaning only: rename columns, cast types, filter deleted rows, convert units. No joins between source tables.

```
-- models/staging/stg_orders.sql {{ config(materialized='view') }} WITH source AS ( SELECT * FROM {{ source('raw', 'orders') }} ), renamed AS ( SELECT id AS order_id, customer_id, CAST(created_at AS TIMESTAMP) AS created_at, UPPER(TRIM(status)) AS status, amount / 100.0 AS amount_usd, -- cents to dollars shipping_address_country AS country_code, _fivetran_synced AS loaded_at FROM source WHERE _fivetran_deleted IS FALSE -- remove soft-deleted rows ) SELECT * FROM renamed
```

2. Intermediate Models

Business logic lives here. Join staging models together. Do not expose to BI tools directly — these feed mart models.

```
-- models/intermediate/int_orders_with_items.sql WITH orders AS ( SELECT * FROM {{ ref('stg_orders') }} ), order_items AS ( SELECT * FROM {{ ref('stg_order_items') }} ), products AS ( SELECT * FROM {{ ref('stg_products') }} ), order_items_enriched AS ( SELECT oi.order_id, oi.product_id, p.product_name, p.category, oi.quantity, oi.unit_price, oi.quantity * oi.unit_price AS line_total FROM order_items oi LEFT JOIN products p ON oi.product_id = p.product_id ) SELECT o.*, i.product_name, i.category, i.line_total FROM orders o LEFT JOIN order_items_enriched i USING (order_id)
```

3. Fact Table Models

```
-- models/marts/fct_orders.sql {{ config(materialized='table') }} WITH orders AS ( SELECT * FROM {{ ref('int_orders_with_items') }} ) SELECT order_id, customer_id, created_at, DATE_TRUNC('month', created_at) AS order_month, status, SUM(line_total) AS order_total, COUNT(DISTINCT product_id) AS distinct_products, CASE WHEN status = 'completed' THEN TRUE ELSE FALSE END AS is_completed FROM orders GROUP BY 1, 2, 3, 4, 5
```

4. Dimension Table Models

```
-- models/marts/dim_customers.sql {{ config(materialized='table') }} WITH customers AS ( SELECT * FROM {{ ref('stg_customers') }} ), orders AS ( SELECT customer_id, MIN(created_at) AS first_order_date, MAX(created_at) AS latest_order_date, COUNT(*) AS total_orders, SUM(order_total) AS lifetime_value FROM {{ ref('fct_orders') }} GROUP BY 1 ) SELECT c.customer_id, c.customer_name, c.email, c.country_code, o.first_order_date, o.latest_order_date, o.total_orders, o.lifetime_value, CASE WHEN o.lifetime_value >= 10000 THEN 'enterprise' WHEN o.lifetime_value >= 1000 THEN 'mid_market' ELSE 'smb' END AS customer_segment FROM customers c LEFT JOIN orders o USING (customer_id)
```

5. Incremental Models (for Large Tables)

Only process new or changed records. Dramatically faster on tables with millions of rows. Use for event logs, clickstream data, transactional logs, sensor data.

```
-- models/marts/fct_events.sql {{ config( materialized='incremental', unique_key='event_id', incremental_strategy='merge', -- merge, append, delete+insert on_schema_change='sync_all_columns' ) }} WITH events AS ( SELECT event_id, user_id, event_type, event_timestamp, properties FROM {{ source('raw', 'events') }} -- On incremental runs: only process new rows {% if is_incremental() %} WHERE event_timestamp > ( SELECT MAX(event_timestamp) FROM {{ this }} ) {% endif %} ) SELECT * FROM events -- Full rebuild: dbt run --full-refresh -s fct_events
```

> ■ WARNING
> WARNING: Always use dbt run --full-refresh -s model_name when you change an incremental model's SQL logic. Otherwise old rows keep the old structure and new rows have the new structure, causing silent data corruption.

Model Config Block Options

```
{{ config( materialized = 'table', -- view, table, incremental, ephemeral schema = 'finance', -- override default schema alias = 'revenue_summary', -- rename in DB tags = ['finance', 'daily'], pre_hook = ["DELETE FROM {{ this }} WHERE date < '2020-01-01'"], post_hook = ["GRANT SELECT ON {{ this }} TO ROLE reporter"], persist_docs = {'relation': true, 'columns': true}, on_schema_change = 'sync_all_columns', -- Snowflake specific: cluster_by = ['order_date', 'customer_id'], -- BigQuery specific: partition_by = {'field': 'order_date', 'data_type': 'date'}, require_partition_filter = true ) }}
```

CHAPTER 5

Declaring Sources

Sources declare raw tables loaded by ingestion tools (Fivetran, Airbyte, Stitch, etc.). Declaring sources gives you freshness checks, lineage tracking, and structured documentation.

```
# models/staging/_sources.yml version: 2 sources: - name: raw # source alias (used in source() calls) database: raw_db # database override (optional) schema: salesforce # raw schema name description: "Salesforce CRM data loaded by Fivetran daily" freshness: # global freshness policy warn_after: {count: 6, period: hour} error_after: {count: 24, period: hour} tables: - name: accounts loaded_at_field: _fivetran_synced description: "Salesforce Account objects" columns: - name: id description: "Salesforce Account ID" tests: [unique, not_null] - name: name description: "Account name" - name: industry tests: - accepted_values: values: ['Technology', 'Finance', 'Healthcare', 'Retail'] - name: opportunities loaded_at_field: _fivetran_synced description: "Salesforce Opportunities (pipeline deals)" freshness: # override at table level warn_after: {count: 12, period: hour}
```

Using Sources in Models

```
-- Only use source() in the very first layer (staging) -- Never reference source() from mart or intermediate models SELECT * FROM {{ source('raw', 'accounts') }}
```

Source Freshness Checks

```
# Check freshness of all sources dbt source freshness # Check a specific source dbt source freshness --select source:raw # Result: GREEN (fresh) / YELLOW (warn) / RED (error) # Fails the job if error threshold exceeded
```

> **TIP**
> Run source freshness as the FIRST step in your production job. If raw data has not landed, fail early before wasting compute on transforms that will produce stale results.

Seeds (CSV Lookup Tables)

Seeds are CSV files in the seeds/ directory that dbt loads directly into your warehouse. Use them for static reference data that changes rarely: country codes, product categories, cost center mappings, etc.

```
# seeds/country_codes.csv country_code,country_name,region,currency US,United States,North America,USD GB,United Kingdom,Europe,GBP DE,Germany,Europe,EUR JP,Japan,Asia Pacific,JPY

# Load seeds to warehouse dbt seed # Reload a specific seed dbt seed --select country_codes # Reference in a model SELECT o.*, c.country_name, c.region FROM {{ ref('fct_orders') }} o LEFT JOIN {{ ref('country_codes') }} c USING (country_code)

# dbt_project.yml — configure seed column types seeds: my_project: country_codes: +column_types: country_code: varchar(2) region: varchar(50) currency: varchar(3)
```

CHAPTER 6

dbt has two types of tests: generic tests (configured in YAML) and singular tests (custom SQL files). Tests return zero rows when passing and return rows when failing.

Generic Tests — Built-in Four

Test	What It Validates
unique	No duplicate values in the column
not_null	No NULL values in the column
accepted_values	All column values are in the allowed list
relationships	Foreign key referential integrity — every value exists in the parent

```
# models/marts/_schema.yml version: 2 models: - name: fct_orders description: "One row per order" columns: - name: order_id tests: [unique, not_null] # primary key - name: customer_id tests: - not_null - relationships: to: ref('dim_customers') field: customer_id - name: status tests: - accepted_values: values: ['pending', 'completed', 'cancelled', 'refunded'] - name: order_total tests: - not_null - dbt_utils.accepted_range: min_value: 0 max_value: 100000
```

Generic Test Configuration

```
columns: - name: amount tests: - not_null: severity: warn # warn instead of failing the run config: where: "status = 'completed'" # conditional: only check completed limit: 100 # show max 100 failing rows store_failures: true # persist failed rows to a table
```

Singular Tests (Custom SQL)

A .sql file in tests/ that returns rows ONLY when the test FAILS. Use for complex business-rule validation that cannot be expressed as generic tests.

```
-- tests/assert_orders_total_positive.sql -- Returns rows when test FAILS (any negative total) SELECT order_id, order_total FROM {{ ref('fct_orders') }} WHERE order_total < 0

-- tests/assert_no_future_orders.sql SELECT order_id, created_at FROM {{ ref('fct_orders') }} WHERE created_at > CURRENT_TIMESTAMP

-- tests/assert_revenue_matches_line_items.sql -- Row-level reconciliation test WITH order_totals AS ( SELECT order_id, order_total FROM {{ ref('fct_orders') }} ), line_item_totals AS ( SELECT order_id, SUM(line_total) AS computed_total FROM {{ ref('fct_order_items') }} GROUP BY 1 ) SELECT o.order_id, o.order_total AS declared_total, l.computed_total AS computed_total FROM order_totals o JOIN line_item_totals l USING (order_id) WHERE ABS(o.order_total - l.computed_total) > 0.01
```

dbt-expectations Package (30+ Extra Tests)

```
# Add to packages.yml first, then dbt deps - package: calogica/dbt_expectations version: [">=0.10.0"] # Usage in schema YAML: - name: email tests: - dbt_expectations.expect_column_values_to_match_regex: regex: "^[^@]+@[^@]+\.[^@]+$" - name: order_date tests: - dbt_expectations.expect_column_values_to_be_between: min_value: "'2020-01-01'" max_value: "current_date" - name: amount tests: -
```

```
dbt_expectations.expect_column_mean_to_be_between: min_value: 10 max_value: 500 - name:
fct_orders # model-level test tests: - dbt_expectations.expect_table_row_count_to_be_between:
min_value: 1000 max_value: 10000000
```

Running Tests

```
dbt test # run all tests dbt test -s fct_orders # test specific model dbt test -s tag:finance #
test models with tag dbt test --select test_type:singular # only singular tests dbt test
--select test_type:generic # only generic tests # Run and test together (recommended for
production) dbt build # runs: seeds + snapshots + models + tests
```

> ■ **HERO STANDARD**
>
> HERO STANDARD: Every model must have unique + not_null on its primary key, relationships tests on every foreign key, and accepted_values on every status or type column. Run dbt build (not just dbt run) to test after every model run.

CHAPTER 7

Inline YAML Documentation

```
# models/marts/_schema.yml version: 2 models: - name: dim_customers description: > One row per unique customer. Includes customer demographics, lifetime value metrics, and engagement history. Primary source is Salesforce CRM with enrichment from Segment events. columns: - name: customer_id description: "Unique customer identifier (sourced from Salesforce)" - name: customer_segment description: | Customer tier based on 12-month lifetime value: - 'enterprise': LTV > $10,000 - 'mid_market': LTV $1,000 - $10,000 - 'smb': LTV < $1,000 - name: lifetime_value description: "Total revenue attributed to this customer (USD)" - name: first_order_date description: "Date of the customer's first completed order"
```

Doc Blocks (Reusable Descriptions)

```
-- docs/column_descriptions.md {% docs customer_id %} The unique identifier for a customer, sourced from Salesforce CRM. This ID is stable across all systems and is the canonical join key for all customer-related data across sources and platforms. {% enddocs %} {% docs order_status %} The current fulfillment status of the order. Possible values: - `pending`: Order placed but not yet processed - `completed`: Order fulfilled and delivered to customer - `cancelled`: Order cancelled before fulfillment - `refunded`: Order was completed but subsequently refunded {% enddocs %}

# Reference doc blocks in your YAML schema: columns: - name: customer_id description: "{{ doc('customer_id') }}" - name: status description: "{{ doc('order_status') }}"
```

Generate & Serve Documentation

```
# Generate the static docs site dbt docs generate # Open in browser (http://localhost:8080) dbt docs serve # The site includes: # - Interactive DAG lineage graph (click any model to explore) # - All model and column descriptions # - Source freshness status # - Compiled SQL for every model # - Test results per model # - Search across all models, columns, sources
```

INFO

In dbt Cloud, docs are automatically generated after each job run and hosted at a shareable URL. Share it with your data team and BI stakeholders as your internal data catalog.

CHAPTER 8

dbt uses Jinja2 templating — a Python-based templating language that adds programming constructs to your SQL: variables, conditionals, loops, and functions (macros).

Jinja Syntax Basics

```
{{ expression }} -- outputs a value {% statement %} -- control flow (if, for, set) {# comment #} -- ignored, not rendered
```

Built-in Variables

Variable	Type	Example Value
target.name	string	'dev' or 'prod'
target.schema	string	'dbt_yourname' or 'public'
target.database	string	'analytics'
target.type	string	'snowflake' or 'bigquery'
this	Relation	analytics.dbt_dev.fct_orders (current model)
model.name	string	'fct_orders'
run_started_at	datetime	2024-01-15 09:30:00 UTC

Conditional Logic

```
-- Environment-specific filtering (dev gets 7 days, prod gets all) {% if target.name == 'prod' %} WHERE is_test_account = FALSE AND created_at >= DATEADD(year, -2, CURRENT_DATE) {% else %} WHERE created_at >= DATEADD(day, -7, CURRENT_DATE) {% endif %}
```

Loops

```
-- Dynamically generate columns from a list {% set payment_methods = ['credit_card', 'paypal', 'bank_transfer', 'crypto'] %} SELECT order_id, {% for method in payment_methods %} SUM(CASE WHEN payment_type = '{{ method }}' THEN amount ELSE 0 END) AS {{ method }}_amount {% if not loop.last %},{% endif %} {% endfor %} FROM {{ ref('stg_payments') }} GROUP BY 1
```

Writing Macros

```
-- macros/cents_to_dollars.sql {% macro cents_to_dollars(column_name, scale=2) %} ROUND({{ column_name }} / 100.0, {{ scale }}) {% endmacro %} -- Usage in any model: SELECT order_id, {{ cents_to_dollars('amount_cents') }} AS amount_dollars, {{ cents_to_dollars('tax_cents', 4) }} AS tax_dollars FROM source

-- macros/safe_divide.sql {% macro safe_divide(numerator, denominator) %} CASE WHEN {{ denominator }} = 0 OR {{ denominator }} IS NULL THEN NULL ELSE {{ numerator }} / NULLIF({{ denominator }}, 0) END {% endmacro %} -- Usage: {{ safe_divide('revenue', 'sessions') }} AS revenue_per_session
```

```
-- macros/generate_schema_name.sql -- Override default schema generation (remove dev schema prefix in prod) {% macro generate_schema_name(custom_schema_name, node) -%} {%- set default_schema = target.schema -%} {%- if custom_schema_name is none -%} {{ default_schema }} {%- else -%} {{ custom_schema_name | trim }} {%- endif -%} {%- endmacro %}
```

Variables

```
# Pass variables at runtime dbt run --vars '{"start_date": "2024-01-01", "environment": "prod"}' # In your model WHERE created_at >= '{{ var("start_date") }}' # With a default value WHERE created_at >= '{{ var("start_date", "2020-01-01") }}' # Define defaults in dbt_project.yml vars: start_date: '2020-01-01' environment: 'dev'
```

CHAPTER 9

Snapshots track how records change over time — implementing Slowly Changing Dimensions Type 2 (SCD2). dbt automatically adds dbt_valid_from and dbt_valid_to columns to record the time window each version of a record was active.

Timestamp Strategy

```
-- snapshots/orders_snapshot.sql {% snapshot orders_snapshot %} {{ config( target_schema = 'snapshots', unique_key = 'order_id', strategy = 'timestamp', updated_at = 'updated_at', -- use this column to detect changes ) }} SELECT * FROM {{ source('raw', 'orders') }} {% endsnapshot %}
```

Check Strategy (No updated_at Column)

```
-- snapshots/customer_snapshot.sql {% snapshot customer_snapshot %} {{ config( target_schema = 'snapshots', unique_key = 'customer_id', strategy = 'check', check_cols = ['email', 'address', 'plan_type'], -- track changes in these ) }} SELECT * FROM {{ source('raw', 'customers') }} {% endsnapshot %} -- Run snapshots: dbt snapshot
```

Snapshot Output Schema

Column	Description
(your columns)	All original columns from the source table
dbt_scd_id	Unique surrogate key for this specific row version
dbt_updated_at	When this row was last compared/updated by dbt
dbt_valid_from	Timestamp when this version became active
dbt_valid_to	Timestamp when this version expired (NULL = currently active)

Querying Snapshots

```
-- Get current state (currently active records) SELECT * FROM {{ ref('orders_snapshot') }} WHERE dbt_valid_to IS NULL; -- Point-in-time: what was the order status on June 1st? SELECT * FROM {{ ref('orders_snapshot') }} WHERE order_id = 12345 AND dbt_valid_from <= '2024-06-01' AND (dbt_valid_to > '2024-06-01' OR dbt_valid_to IS NULL); -- Full history of changes for a record SELECT * FROM {{ ref('customer_snapshot') }} WHERE customer_id = 'ABC123' ORDER BY dbt_valid_from;
```

CHAPTER 10

Exposures

Exposures document downstream consumers of your dbt models — dashboards, ML models, apps, APIs. They appear in your lineage graph so you can see what breaks if you change a model.

```
# models/exposures.yml version: 2 exposures: - name: revenue_dashboard type: dashboard # dashboard, analysis, ml, application, notebook maturity: high # low, medium, high url: https://company.looker.com/dashboards/42 description: > Executive revenue dashboard. Shows MRR, ARR, and churn. Updated daily at 6am UTC. Critical for board reporting. depends_on: - ref('fct_orders') - ref('dim_customers') - ref('agg_monthly_revenue') owner: name: Revenue Analytics Team email: revenue@company.com - name: churn_ml_model type: ml maturity: medium description: "30-day churn probability model trained on customer events" depends_on: - ref('fct_customer_events') - ref('dim_customers') owner: name: Data Science email: datascience@company.com
```

Metrics Layer — dbt Semantic Layer (dbt Cloud)

Define business metrics once in dbt, query them consistently from any BI tool. Powered by MetricFlow. Available in dbt Cloud with supported adapters.

```
# models/semantic/sem_orders.yml version: 2 semantic_models: - name: orders description: "Orders semantic model" model: ref('fct_orders') entities: - name: order type: primary expr: order_id - name: customer type: foreign expr: customer_id dimensions: - name: order_date type: time expr: created_at type_params: {time_granularity: day} - name: status type: categorical measures: - name: order_total agg: sum expr: order_total - name: order_count agg: count_distinct expr: order_id metrics: - name: monthly_revenue label: Monthly Revenue type: simple type_params: measure: order_total filter: "{{ Dimension('order__status') }} = 'completed'" - name: avg_order_value label: Average Order Value type: ratio type_params: numerator: order_total denominator: order_count
```

CHAPTER 11

Packages are reusable collections of models, macros, and tests published at hub.getdbt.com. Think of them as npm for your SQL transformation layer.

Installing Packages

```
# packages.yml packages: - package: dbt-labs/dbt_utils version: [">=1.0.0", "<2.0.0"] - package: calogica/dbt_expectations version: [">=0.10.0", "<0.11.0"] - package: calogica/dbt_date version: [">=0.10.0"] - package: dbt-labs/codegen version: [">=0.12.0"] - package: dbt-labs/audit_helper version: [">=0.12.0"] - package: elementary-data/elementary version: [">=0.14.0"] - package: fivetran/salesforce version: [">=0.9.0"] # Install all packages: dbt deps
```

Essential Packages Reference

Package	What It Provides
dbt_utils	50+ macros: surrogate_key, star, union_relations, deduplicate, date_spine, safe_cast
dbt_expectations	30+ Great Expectations-style tests: regex, ranges, row counts, column stats
dbt_date	Date spine generation, fiscal calendars, time-zone conversions
codegen	Auto-generate YAML schema files from existing models (huge time saver)
audit_helper	Compare model results before/after refactoring — safe migrations
elementary	Data observability: anomaly detection, schema change monitoring, Slack alerts
fivetran/stripe	Pre-built models for Stripe: MRR, churn, revenue recognition
fivetran/salesforce	Pre-built models for Salesforce: opportunities, accounts, contacts

Key dbt_utils Macros

```
-- 1. Surrogate key from multiple columns {{ dbt_utils.generate_surrogate_key(['order_id', 'product_id']) }} -- 2. Select all columns except specific ones {{ dbt_utils.star(from=ref('stg_orders'), except=["_fivetran_synced", "_fivetran_deleted"]) }} -- 3. Date spine (generate one row per day) {{ dbt_utils.date_spine( datepart="day", start_date="cast('2022-01-01' as date)", end_date="current_date" ) }} -- 4. Deduplicate keeping latest row {{ dbt_utils.deduplicate( relation=source('raw', 'orders'), partition_by='order_id', order_by='updated_at desc' ) }} -- 5. Union multiple relations {{ dbt_utils.union_relations( relations=[ref('orders_us'), ref('orders_eu'), ref('orders_apac')] ) }} -- 6. Safe add months (handles date edge cases) {{ dbt_utils.dateadd('month', 3, 'subscription_start_date') }} -- Auto-generate schema YAML (codegen): dbt run-operation codegen.generate_model_yaml --args '{model_name: fct_orders}'
```

Feature	dbt Core	dbt Cloud
Cost	Free, open source	Free tier + paid plans ($50+/mo/seat)
Scheduling	External (Airflow, Cron, etc)	Built-in scheduler + webhook triggers
IDE	Local editor (VS Code)	Browser-based cloud IDE with autocomplete
CI/CD	GitHub Actions (DIY setup)	Native Slim CI — auto on every PR
Docs hosting	Self-host	Auto-hosted, shareable URL
Semantic Layer	MetricFlow CLI only	Full MetricFlow with BI integrations
Environments	Manual target management	Dev / Staging / Prod UI with isolation
Audit log	No	Full audit trail of all runs
SSO / RBAC	No	Enterprise — SAML SSO, group-level RBAC
Job orchestration	External only	Job dependencies, triggers, DAG of jobs
Run history	Local target only	Full run history with artifacts stored
Collaboration	Git-based only	Browser IDE + shared environments

■ INFO

Recommendation: Start with dbt Core locally. Upgrade to dbt Cloud when your team grows past 2-3 engineers, when you need built-in CI/CD, or when stakeholders need self-serve documentation and scheduled job monitoring.

dbt Cloud Job Configuration

```
# Typical production job commands (in dbt Cloud UI): dbt source freshness # Step 1: fail if data is stale dbt build --target prod # Step 2: run + test everything dbt docs generate # Step 3: refresh documentation # Slim CI job (runs on every PR): dbt build \ --select state:modified+ \ --defer \ --state gs://your-bucket/prod-artifacts/ \ --target ci
```

CHAPTER 13

The Medallion Architecture with dbt

Layer	dbt Folder	Materialization	Description
Bronze / Raw	N/A (sources)	External tables	Raw ingested data — untouched
Silver / Staging	models/staging/	View	Cleaned, renamed, typed
Gold / Marts	models/marts/	Table	Business-ready facts & dimensions

Model Contracts (dbt 1.5+)

Contracts enforce that a model's output matches its declared schema before materializing. Prevents silent schema drift.

```
models: - name: dim_customers config: contract: enforced: true # dbt checks schema before writing columns: - name: customer_id data_type: integer constraints: - type: not_null - type: primary_key - name: email data_type: varchar constraints: - type: not_null - name: lifetime_value data_type: numeric
```

Unit Testing (dbt 1.8+)

Test model logic with mock data — no warehouse query needed for fast iteration.

```
# models/tests/unit/test_customer_segment.yml unit_tests: - name: test_customer_segment_logic model: dim_customers given: - input: ref('stg_customers') rows: - {customer_id: 1, lifetime_value: 15000} # enterprise - {customer_id: 2, lifetime_value: 5000} # mid_market - {customer_id: 3, lifetime_value: 200} # smb expect: rows: - {customer_id: 1, customer_segment: 'enterprise'} - {customer_id: 2, customer_segment: 'mid_market'} - {customer_id: 3, customer_segment: 'smb'} # Run unit tests: dbt test --select test_type:unit
```

Advanced Incremental Strategies

```
{{ config( materialized='incremental', unique_key=['order_id', 'product_id'], -- composite key incremental_strategy='merge', -- Snowflake / BQ / Databricks merge_exclude_columns=['inserted_at'], -- never overwrite these on_schema_change='sync_all_columns', -- handle new columns partition_by={'field': 'order_date', 'data_type': 'date'}, -- BQ only cluster_by=['customer_id'] -- Snowflake clustering ) }}
```

Multiple Schemas & Cross-Database

```
{{ config( database='analytics_prod', -- write to a specific database schema='finance', -- specific schema alias='revenue_by_month' -- custom table name ) }} -- Creates: analytics_prod.finance.revenue_by_month -- Cross-database reference (advanced): SELECT * FROM {{ ref('dim_customers', database='shared_db') }}
```

Pre/Post Hooks

```
{{ config( pre_hook=[ -- Run before model executes "ALTER TABLE {{ this }} CLUSTER BY (order_date)", "DELETE FROM {{ this }} WHERE order_date < DATEADD(year, -3, CURRENT_DATE)" ],
```

```
post_hook=[ -- Run after model executes "GRANT SELECT ON {{ this }} TO ROLE reporter", "GRANT
SELECT ON {{ this }} TO ROLE analyst", "ANALYZE {{ this }}" ] ) }}
```

CHAPTER 14

GitHub Actions CI Pipeline

```
# .github/workflows/dbt_ci.yml name: dbt CI on: pull_request: branches: [main] paths: ['models/**', 'tests/**', 'macros/**', 'dbt_project.yml'] jobs: dbt-ci: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-python@v4 with: python-version: '3.11' - name: Install dbt run: pip install dbt-snowflake - name: Install packages run: dbt deps - name: Download prod artifacts (for Slim CI) run: | dbt ls --target prod --profiles-dir . > /dev/null # prime artifacts # Or download from S3/GCS where your prod job uploads them - name: dbt Slim CI build env: DBT_USER: ${{ secrets.DBT_USER }} DBT_PASSWORD: ${{ secrets.DBT_PASSWORD }} run: | dbt build \ --select state:modified+ \ --defer \ --state ./prod-artifacts \ --target ci \ --profiles-dir . - name: Upload CI artifacts uses: actions/upload-artifact@v4 if: always() with: name: dbt-ci-artifacts path: target/
```

Slim CI Selectors

Selector	Meaning
state:modified	Models with changed SQL, YAML config, or dependencies
state:modified+	Modified models + ALL their downstream children
state:new	Brand new models not in the prod state
state:modified.body	Models where the SQL body itself changed (not just config)

Production Deployment Checklist

1	**Source freshness check**	dbt source freshness — fail early if data hasn't landed
2	**dbt build (not dbt run)**	Runs seeds → snapshots → models → tests in dependency order
3	**Correct test severities**	error for critical tests, warn for informational checks
4	**Grant permissions**	Use post_hook to grant SELECT to reporting roles
5	**Generate documentation**	dbt docs generate — keeps catalog always current
6	**Upload artifacts**	Store target/ to S3/GCS so Slim CI can diff against it

Orchestration with Airflow (Astronomer Cosmos)

```
from astronomer.cosmos import DbtTaskGroup, ProjectConfig, ProfileConfig from airflow.decorators import dag from datetime import datetime @dag(schedule="@daily", start_date=datetime(2024, 1, 1)) def dbt_pipeline(): DbtTaskGroup( project_config=ProjectConfig("/path/to/dbt/project"), profile_config=ProfileConfig( profile_name="my_project", target_name="prod", ), operator_args={ "dbt_cmd_flags": ["--full-refresh"], } ) dbt_pipeline()
```

CHAPTER 15

■ CORE PHILOSOPHY

The Analytics Engineer's Manifesto: Transform raw data into trusted, documented, tested assets. Write SQL that reads like documentation. Make data breakage impossible to hide. Treat your SQL like software.

Architecture Checklist

✓ Project Structure

• Staging layer: one model per source table, named stg_sourcename__tablename

• Intermediate layer: business logic only, prefixed int_

• Mart layer: fct_ (facts) and dim_ (dimensions) tables for BI

• Custom schema per layer — staging in staging schema, marts in public

• All raw source tables declared in _sources.yml files

• Seeds for all static lookup/reference data

✓ Quality Gates

• Every model has a YAML schema file with descriptions

• Primary keys: unique + not_null tests on every model

• Foreign keys: relationships tests between fact and dimension tables

• Status/type columns: accepted_values tests on every one

• Source freshness configured for all ingestion sources

• Custom singular tests for critical business rules

• dbt build (not dbt run) so tests always run after models

✓ Code Quality

• No hardcoded schema or database names — always use ref() and source()

• All SQL uses CTEs (no nested subqueries)

• First CTE = source reference, last CTE = final SELECT statement

• DRY macros for repeated logic (date parsing, surrogate keys, safe divide)

• Environment-aware logic with target.name (dev gets small data, prod gets all)

• is_incremental() used in all large event/log tables

• No secrets in code — use env_var() and environment variables

✓ Team & Process

• Every PR triggers slim CI — only test modified + downstream models

• Docs auto-generated after each prod run, shared with stakeholders

• Exposures declare all downstream dashboards, ML models, and apps

• dbt_utils + dbt_expectations installed as standard packages

• Codegen used to bootstrap YAML schemas (never write YAML manually for every column)

• audit_helper used when refactoring existing models (compare before/after)

Quick Reference Card

Task	Command
Run everything in order	dbt build
Run model + ancestors	dbt run -s +my_model
Run model + descendants	dbt run -s my_model+
Run by tag	dbt run --select tag:finance
Full rebuild incremental model	dbt run --full-refresh -s my_model
Test one model	dbt test -s my_model
Check source freshness	dbt source freshness
Compile (no run)	dbt compile -s my_model
View compiled SQL	cat target/compiled/.../my_model.sql
Open documentation	dbt docs generate && dbt docs serve
Generate YAML schema	dbt run-operation codegen.generate_model_yaml --args '{model_name: fct_ord
Run only changed models (CI)	dbt build --select state:modified+ --defer --state ./prod
Install packages	dbt deps
Debug connection	dbt debug

30-Day Learning Path to dbt Hero

Week 1	**Foundations**	Set up dbt Core with DuckDB locally (no cloud needed). Build 3 staging models from CSV seeds. Add unique + not_null tests. Generate and explore docs. Run dbt build.
Week 2	**Intermediate**	Connect to a real warehouse. Build a full staging → intermediate → mart pipeline. Write custom singular tests for business rules. Create your first macro. Add source declarations with freshness checks.

Week 3	**Production-Ready**	Set up incremental models for large tables. Add snapshots for SCD Type 2. Set up GitHub Actions CI with Slim CI. Install dbt_utils and dbt_expectations. Add model contracts to critical mart tables.
Week 4	**Hero Level**	Add exposures for all downstream dashboards. Configure the metrics/semantic layer. Write unit tests for complex transformation logic. Set up elementary for data observability. Present your DAG lineage to stakeholders. Achieve 100% model documentation coverage.

■ YOU ARE NOW A dbt HERO

You now know the complete dbt ecosystem — from installation to advanced CI/CD, from simple views to incremental models and SCD snapshots. The best way to solidify this knowledge is to take your company's most painful manual SQL report and rebuild it end-to-end with dbt: staging → intermediate → mart → tests → docs → CI. The trust, the insights, and the praise will follow.